Monolingual Transformer LMs in Hindi and Nepali

Consolidated Final Report — Phases 1–3

Language Models and Agents, Monsoon 2026 — Individual Project

Model H = Hindi (higher-resource) Model L = Nepali (lower-resource) Branch: `phase-3`

Two independently trained, from-scratch, decoder-only Transformer LMs (∼25M params each), one per language. No pretrained weights, tokenizers, or shared vocabulary between them. Every number below is generated from metric JSON the pipeline scripts produced. Full per-phase detail: `report/phase1_report.md`, `phase2_report.md`, `phase3_report.md`.

1 Phase 1 — Data Collection and Tokenizer Construction

Language choice. Model H = Hindi: best-represented Indian language in IndicCorpV2 (3 shards) with a usable Wikipedia dump, giving headroom to spend the manual lane on source diversity. Model L = Nepali: IndicCorpV2 ships it as a single file, no usable Wikisource wiki exists, forcing a news-only manual lane — a direct, measurable resource gap. Both share Devanagari script, so H-vs-L differences are attributable to resource level and language, not script.

Pipeline. Identical 12-stage streaming pipeline for both languages, zero shared code/data/tokenizer beyond generic utilities in `shared/`: download → normalize → script-filter → quality-filter → dedup (exact + MinHash/LSH) → corpus stats → split (90/5/5, frozen val/test) → decontaminate train against frozen val/test → concatenate → train tokenizer → evaluate tokenizer → tokenize. Val/test are frozen *before* the tokenizer is trained and before decontamination, so held-out perplexity in Phase 2 is on genuinely unseen text.

	Model H (Hindi)	**Model L (Nepali)**
Final train tokens (content)	632,352,544	637,535,988
Manual-lane tokens (% of train)	114,417,599 (18.1%)	116,842,022 (18.3%)
Manual sources	Wikipedia + 3 news outlets	2 news outlets (WordPress API)
Unique whitespace words	3,956,492	6,800,829
Vocabulary size (selected)	12,000	12,000
Tokenizer fertility (tok/word)	**1.3042**	**1.4394**
UNK rate (all vocab sizes)	0.0000	0.0000

Table 1: Phase 1 headline numbers. Both languages exceed the ∼500M-token and ≥20% manual-fraction targets; no shortfall justification is needed for either.

Vocabulary size (12,000, both languages). Fertility falls monotonically with vocab size in both languages, so no size is optimal on compression alone; 12K is where the *marginal* fertility gain per 2,000 added rows starts shrinking, for both languages simultaneously, and keeping the same size for H and L keeps Phase 2 perplexity comparisons uncontaminated by a vocabulary-size confound.

Nepali's higher fertility is morphology, not corpus quality. Nepali is agglutinative (postpositions fuse onto stems, e.g. नेपालको = नेपाल + को), which is why it has *more* unique surface forms (6.80M vs. 3.96M) despite being the lower-resource language, and why BPE needs 10.4% more tokens per word to cover it. This single fact is the root cause traced through Phases 2–3 below.

Requirement	**Hindi**	**Nepali**	**Status**
∼500M training tokens	632,352,544	637,535,988	**PASS** — both exceed by 26–28%
IndicCorp sub-target 400M	517,934,945	520,693,966	**PASS**
Manual sub-target ≥100M	114,417,599 (+14.4%)	116,842,022 (+16.8%)	**PASS**
Tokenizer trained from scratch	yes	yes	**PASS**
No shared tokenizer/vocab/data	yes	yes	**PASS**
Train decontaminated vs. val/test	yes	yes	**PASS**

Table 2: Phase 1 requirement compliance, both languages.

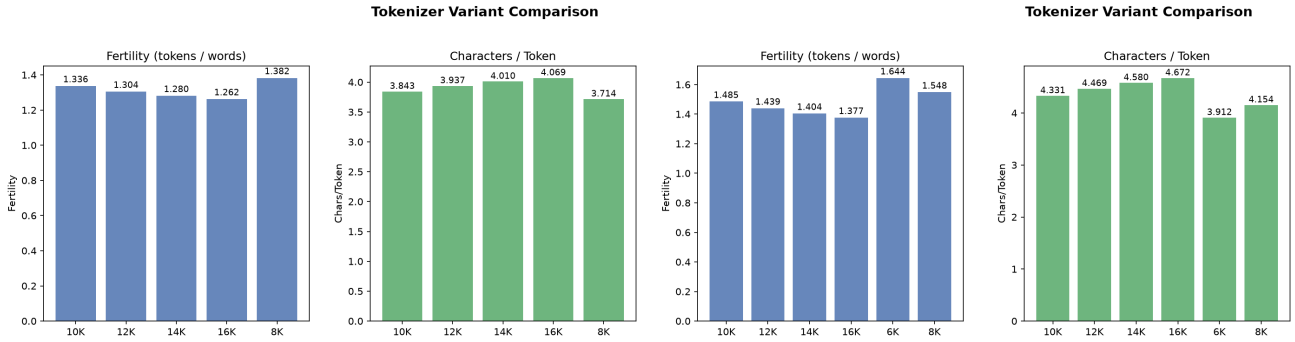


Figure 1: Fertility and chars/token across candidate vocabulary sizes, held-out validation. Hindi (left) is consistently more compressible than Nepali (right) at every matched vocabulary size.

2 Phase 2 — Model, Pretraining, and Evaluation

Architecture. Decoder-only Transformer built *only* from `nn.Linear`/`nn.Embedding`/`nn.LayerNorm`/`nn.Dropout` — no `nn.Transformer*`, no `nn.MultiheadAttention`, no HuggingFace, no fused SDPA. 6 layers, 8 heads, $d_{\text{model}} = 512$, $d_{ff} = 2048$, block size 512, learned absolute positions, pre-norm, GELU, weight tying. **25,321,472** parameters, identical for both languages (weight tying saves 6.14M params = 24.3% of the total). Causal masking verified bit-exact (max $|\Delta| = 0$ at all positions $\leq t$ under a future-token perturbation, both models).

Metric	Model H (Hindi)	Model L (Nepali)
Training tokens (1 epoch)	648,012,574	659,499,440
Optimizer steps	13,183	13,417
Validation perplexity	26.21	41.52
Validation bits-per-byte (BPB)	0.4797	0.4635
Test perplexity / BPB	26.19 / 0.4791	41.46 / 0.4632
Mean attention entropy (nats)	1.3109	1.2671
Mean attention distance (tokens)	3.98	3.84

Table 3: Phase 2 headline results, validation and test.

The central finding: the H/L gap reverses sign depending on the unit. In token space, Model L looks 58.4% worse (PPL 41.52 vs. 26.21). In byte space (BPB, tokenizer-independent), Model L is 3.4% *better* (0.4635 vs. 0.4797) — confirmed on the held-out test split (+58.3% / −3.3%), so it is not a validation-set artifact. **Perplexity is denominated in each model's own tokenizer**: because Nepali's fertility is 10.4% higher, Model L sees less linguistic content per fixed-size training batch, making its per-token prediction problem intrinsically harder — a tokenizer/morphology artifact, not a resource-level modelling failure. This is exactly the reversal the BPB metric exists to catch, and it recurs, restated, in Phase 3's reasoning results (§3).

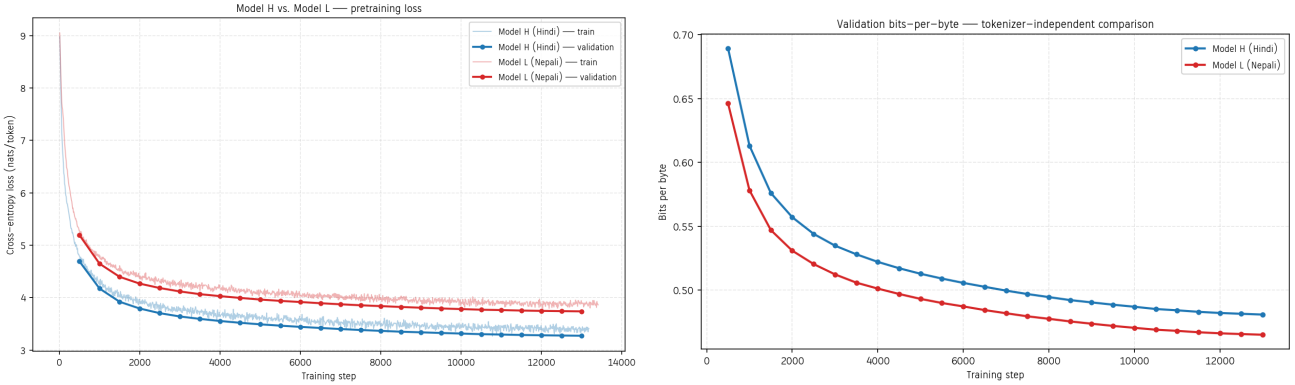


Figure 2: Left: training loss, both models, smooth convergence with no divergence. Right: validation BPB over training — the Model L < Model H ordering holds from step 500 onward, not just at the final checkpoint.

Generation quality. 500 held-out prefixes, greedy + temperatures 0.5/1.0/1.5. BLEU-4 is near-uninformative for open-ended continuation against a single reference (scores <1, occasionally 0); chrF (character n-gram overlap)

is the most informative, rising monotonically with temperature (H: 7.16→18.44; L: 6.71→20.84) since it credits correct stems under different inflections. Greedy decoding shows severe repetition (rate 0.70 H / 0.76 L) that overlap metrics barely penalize; Distinct-2 rises to ∼0.95 for both by T=1.5 as repetition collapses to near zero — no single metric captures quality alone. Both models produce grammatically well-formed, register-appropriate Devanagari text (correct postpositions and verb agreement) but degrade in coherence over longer spans, as expected at 25M/650M-token scale.

Attention. Both models show a mix of local-sharp (positional) and distant-sharp (content-based) heads, classified against each model's own median entropy/distance. Model L's attention is measurably sharper and more local (entropy 1.267 vs. 1.311; distance 3.84 vs. 3.98 tokens) — consistent with its higher fertility: more of the predictive signal for a fragmented word sits in the immediately adjacent subword piece.

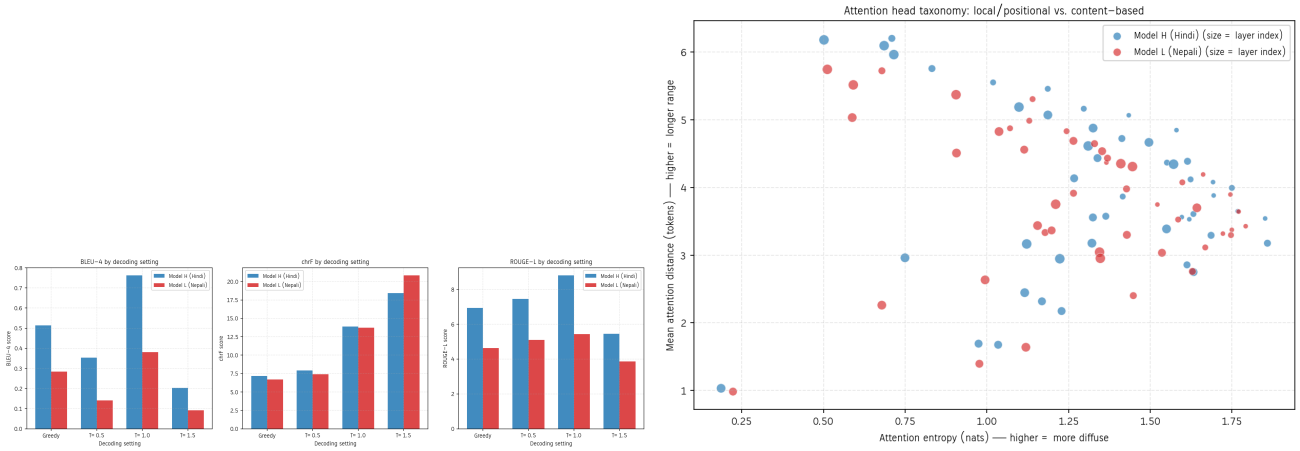


Figure 3: Left: generation-quality metrics (BLEU-4, chrF, ROUGE-L) across decoding settings, both models. Right: local/positional vs. content-based head-taxonomy split, both models, 6 layers × 8 heads each.

3 Phase 3 — Reasoning Finetuning, Attention Analysis

Synthetic reasoning dataset. Own synthetic dataset per language (not downloaded), four attribute domains (age, height, price, quantity) × two task types (pairwise; 3-entity transitive chains) × four paraphrased templates (3 trained on, 1 held out entirely for wording generalization). 2,400 train / 320 each of validation, test_seen, test_unseen_entity, test_unseen_wording, per language. **Leakage control**: each domain's entity pool is split once into disjoint train/test name sets before any example is generated; `test_seen` draws only train names, the two unseen splits draw only test names, independently of the held-out-template axis.

Finetuning protocol. Each model finetuned independently from its own Phase 2 `best.pt`, tokenizer/vocabulary fixed, loss masked to the answer span only. Identical recipe both languages: LR $10^{-4} \rightarrow 10^{-5}$ (cosine, 5% warmup), 4 epochs, batch 16, AdamW (0.9, 0.95), seed 42, 600 total steps (∼12 min Hindi / ∼25 min Nepali wall-clock, CPU). This recipe (LR/epochs lowered, entity pools enlarged 24→80 names) is a direct fix for a first-round run that scored 0.422 on `test_seen` but exactly 0.000 on both unseen splits — the course's own memorization case study reproduced almost exactly; the fix is documented and the corrected numbers below are the retrained result.

Test split	Model	Pretrained	Finetuned	Δ
Seen entity + seen wording	H (Hindi)	0.000	0.412	+0.412
Unseen entity + seen wording	H (Hindi)	0.000	0.066	+0.066
Unseen entity + unseen wording	H (Hindi)	0.000	0.059	+0.059
Seen entity + seen wording	L (Nepali)	0.000	0.434	+0.434
Unseen entity + seen wording	L (Nepali)	0.000	0.191	+0.191
Unseen entity + unseen wording	L (Nepali)	0.000	0.203	+0.203

Table 4: Three-way reasoning exact-match accuracy, pretrained-only vs. finetuned (mean over 3 seeds; greedy decoding, so seeds are a correctness check, not a variance estimate).

Notable reversal. Model L (Nepali) generalizes *more* reliably than Model H (Hindi) on both held-out axes (unseen-entity 0.191 vs. 0.066; unseen-wording 0.203 vs. 0.059), despite being the lower-resource model throughout Phases 1–2. This is the opposite direction from every raw token-level Phase 1–2 metric. Per the SFT format-vs-substance

distinction, this is read as a difference in what each pretrained base could already do latently, not as evidence Nepali's language modelling is stronger (Phase 2 showed the opposite on perplexity, and BPB showed the two nearly tied).

	Before SFT	After SFT
Model H pretraining-val PPL	26.38	38.55
Model L pretraining-val PPL	42.32	65.47

Table 5: Catastrophic-forgetting diagnostic: PPL on the *original* Phase 2 pretraining validation set, before/after reasoning SFT. Expected to rise (narrow synthetic distribution); this measures distribution shift, not reasoning success.

Error provenance (quantified, not just illustrated). Every wrong prediction on both unseen splits was classified programmatically against each language's own train/test entity pools:

Lang.	Task	n wrong	Train-pool halluc.	Malformed	Legal wrong entity	Empty/other
H	pairwise	291	77.3%	9.3%	9.3%	4.1%
H	transitive3	309	85.4%	3.6%	4.9%	6.1%
L	pairwise	240	58.8%	8.3%	25.4%	7.5%
L	transitive3	274	47.4%	2.2%	44.9%	5.5%

Table 6: Error-provenance classification (`shared/eval/error_provenance.py`). Hindi's errors are dominated by hallucinating a training-pool name (memorization); Nepali more often names the *other* real entity in the prompt (*legal but wrong entity*, 25–45%) rather than hallucinating — a genuinely different, more prompt-engaged failure mode, consistent with Nepali's stronger unseen-split accuracy above.

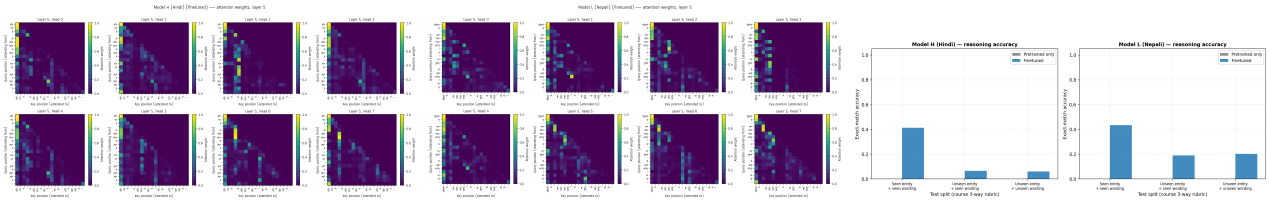


Figure 4: Left/middle: late-layer (L5) post-finetune attention heatmaps, Hindi and Nepali, on the same comparative-reasoning prompt structure — the pretrained global first-token sink is replaced by a content-specific anchor in both languages after finetuning. Right: three-way accuracy comparison, both models.

Finetuning **sharpens** late-layer attention (mean entropy: H 1.5936 → 1.5585, L 1.6193 → 1.6163) while lengthening mean attention distance slightly (H 4.86 → 5.02, L 4.95 → 5.08 tokens) — consistent with heads reorganizing toward a fixed content anchor rather than a purely positional pattern, in both languages independently.

4 Final Comparison — the Four Required Questions

1. How did data scale and quality differ between Model H and Model L? Scale is near-identical (648.0M vs. 659.5M train tokens; both exceed the 500M target by 26–28%, both clear the ≥20% manual floor) — *not* the differentiator. The real asymmetry is morphological: Nepali's agglutinative case-marking produces 6.80M unique surface forms against Hindi's 3.96M at comparable corpus size, and 10.4% higher tokenizer fertility (1.4394 vs. 1.3042).

2. How do language-modeling and reasoning results compare across the two resource tiers? Phase 2: Model L is 58.4% worse on perplexity but 3.4% *better* on BPB — the token-space gap is a tokenizer artifact. Phase 3 reasoning accuracy is the first metric in this project measuring genuinely learned behavior rather than distributional fit, and it does *not* favor Model H either: Model L reaches 0.191/0.203 on unseen-entity/unseen-wording against Model H's 0.066/0.059. Across three independent measurement axes (perplexity, BPB, reasoning generalization), only the token-space perplexity metric favors the higher-resource model — the other two either favor Model L or are near-tied.

3. What tokenizer/corpus factors most affected the lower-resource model? Fertility is the dominant, well-evidenced factor for Phase 1–2 results (BPB reversal). It does *not*, however, explain the Phase 3 `transitive3` Hindi-vs-Nepali gap on the higher-resource model's own error profile: the one concrete tokenizer-level hypothesis tested for that gap (inconsistent subword segmentation of Nepali's glued comparative भन्दा) was checked directly against the tokenizer and refuted. This is reported as an open question, not force-fit to the fertility story that explains the unrelated Phase 2 result.

4. What evidence explains the observed differences? Direct inspection of per-example predictions (not aggregate accuracy alone) is what actually explains the failure modes: Hindi's wrong answers are dominated by clean hallucinations of familiar training-pool names (77–85%), evidence of entity-identity memorization rather than genuine comparison; Nepali more often names the *other* real entity in the prompt (25–45% "legal but wrong entity"), evidence it is at least reading the prompt's own entities even when it errs. The attention comparison adds a mechanistic layer: finetuning sharpens late-layer attention onto content-specific positions (replacing the pretrained attention sink) in both languages independently, which is direct evidence of *where* each model's internal computation changed, not just that its outputs did.

5 Where Automation Substitutes for Human Judgment

- **No fluent reader assessed retained/rejected corpus text (Phase 1).** Script-filter ratios, quality heuristics, and the 0.4 near-duplicate similarity threshold are conventional/algorithmic choices, not tuned against sampled Hindi/Nepali text.
- **Generation quality (Phase 2) was scored by metrics, not a fluent reader**; qualitative examples are shown as evidence of grammaticality only.
- **Attention head labels (Phases 2–3) are statistical** (median-split on entropy/distance), not interpretively verified against what a head actually attends to linguistically.
- **Reasoning template naturalness (Phase 3)** was reviewed by a fluent speaker for the 12 distinct sentence patterns per language (not all ∼6,000 rendered examples individually), and one real Nepali compounding bug was found and fixed before the final finetuning run reported above.
- **The Hindi/Nepali `transitive3` generalization gap** is reported with one tested-and-refuted tokenizer hypothesis and no confirmed replacement cause — an honest open question, not a manufactured explanation.

6 Reproduction

```
# Phase 1 (per language; replace hindi -> nepali for Model L)
python hindi/scripts/01_download_indiccorp.py  ...  python hindi/scripts/12_tokenize_corpus.py
  # (stages 01-12 in order; see hindi/Hindi_Corpus_Pipeline_Run_Instructions.md)
# Phase 2
python hindi/train/pretrain.py                       # resumes automatically from checkpoint
python hindi/eval/evaluate_lm.py --checkpoint hindi/checkpoints/best.pt
python hindi/eval/evaluate_generation.py --checkpoint hindi/checkpoints/best.pt
python hindi/eval/analyze_attention.py --checkpoint hindi/checkpoints/best.pt
# Phase 3
python hindi/reasoning_data/generate_dataset.py
python hindi/finetune/finetune.py                     # resumes automatically
python hindi/eval/evaluate_reasoning.py --device cpu  # 3-way accuracy + PPL before/after
python hindi/eval/compare_attention.py --device cpu
python report/scripts/make_phase2_figures.py && python report/scripts/make_phase3_figures.py
python report/scripts/build_phase2_report.py && python report/scripts/build_phase3_report.py
python -m pytest tests/test_phase2.py tests/test_phase3.py -v   # 22 + 23 correctness gates
```

Full per-phase step-by-step instructions, exact config keys, and all Google Drive links (raw/final corpora, pretrained checkpoints, finetuned checkpoints, both languages) are in the top-level `README.md`. Per-phase detail, all figures, and complete tables live in `report/phase1_report.md`, `phase2_report.md`, `phase3_report.md`.